

Data Structure Lab. Project #1(설계)

2026 년 9 월 14 일

Due date: 2026 년 10 월 11 일 일요일 23:59:59 까지

본 프로젝트에서는 이진 탐색 트리(Binary Search Tree), 연결 리스트(Linked List), 큐(Queue) 자료 구조를 이용하여 나만의 운동 루틴 프로그램을 구현한다. 이 프로그램은 운동 데이터가 저장되어 있는 데이터 파일로부터 운동 종목, 장비, 소요 시간, 칼로리 정보를 읽어 Queue 를 구축하며, 해당 Queue 를 WorkoutQueue 라 부른다. Queue 에서 QPOP 명령어를 실행하면 데이터를 방출하여 운동 종목을 기준으로 정렬된 자료구조(ExerciseBST)와 장비를 기준으로 정렬된 자료구조(EquipmentBST)에 저장한다. 사용자가 루틴 생성 명령어를 실행하면, 두 BST 에 저장된 운동 정보를 기반으로 양방향 연결리스트를 구성하여 실제 운동 루틴을 완성한다.

본 프로젝트에서 데이터 한 줄은 하나의 훈련 세트를 의미한다. 즉 "이 운동을, 이 장비로, 이만큼 수행한다"는 뜻이며, 같은 운동이라도 장비가 다르면 소요 시간과 칼로리가 다른 서로 다른 세트로 취급한다.

- 운동 종목을 기준으로 정렬된 자료구조

ExerciseBST 는 운동 종목, 해당 운동으로 가능한 장비, 각 세트의 소요 시간과 칼로리로 구성되며, 운동 종목 이름을 기준으로 정렬된다. 각 노드에는 해당 운동의 모든 세트 정보가 저장된다.

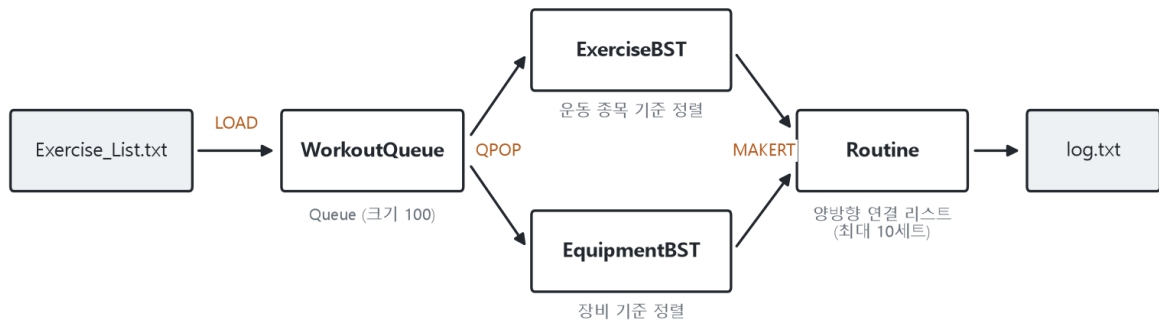
- 장비를 기준으로 정렬된 자료구조

EquipmentBST 는 장비 이름, 해당 장비로 할 수 있는 운동 종목들, 각 세트의 소요 시간과 칼로리로 구성되며, 장비 이름을 기준으로 정렬된다. 각 노드에는 동일한 장비를 사용하는 모든 세트 정보가 저장된다. 두 BST 는 서로 다른 데이터를 담는 것이 아니라, 동일한 세트 집합을 운동 기준과 장비 기준으로 각각 정렬해 보관한다. 따라서 하나의 세트는 항상 두 BST 에 함께 존재한다.

- BST 로부터 데이터로 이뤄진 자료구조

Routine 은 BST 에 저장된 운동 데이터를 기반으로 구축되며, 양방향 연결 리스트 형태로 구성된다. 각 노드는 단일 세트에 대한 정보(운동 종목, 장비, 소요 시간, 칼로리)로 구성되며, 앞뒤 양쪽 끝에서 세트를 추가하거나 제거할 수 있다.

각 자료구조의 구축 방법과 조건에 대한 자세한 설명은 Program Implementation 에서 설명한다.

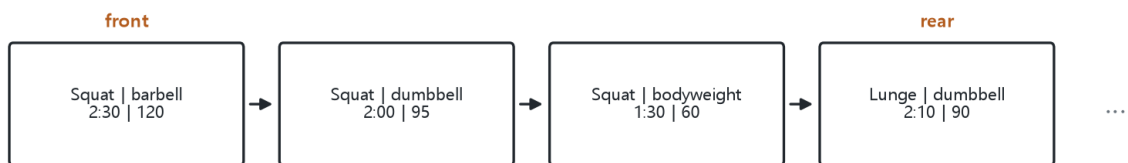


[그림 1] 프로그램 구조

□ Program implementation

1) WorkoutQueue

- 주어진 Exercise_List.txt 에 저장된 운동 데이터를 이용하여 구축한다. Queue 의 자료구조에 따라 저장되는 순서대로 방출(FIFO)된다. Queue 에는 Exercise_List.txt 의 첫 번째 줄부터 마지막 줄까지 순서대로 저장되며, **이때 동일한 세트의 중복은 존재하지 않는다.**
- 입력 데이터는 "운동 종목 | 장비 | 소요 시간 | 칼로리"(ex. Squat|barbell|2:30|120)로 구성되며, '|' 문자를 기준으로 데이터가 나뉜다. **이때 운동은 대문자로 시작하고, 장비는 소문자로 구성되며,** Exercise_List.txt 에서 불러온 세트가 아닌 새로 세트를 추가하는 경우 Queue 에 이미 존재하는지 확인해야 한다.
- Queue 를 구성하는 데이터는 WorkoutQueueNode class 로 구현되며, 각 노드는 단일 세트의 형태이다.
- Queue 의 크기는 100 으로 초기화되고 이후 크기가 변하지 않으며, Queue 가 비어 있을 때 POP 또는 전부 차 있을 때 PUSH 가 수행되는 경우 에러 코드를 출력한다.
- Queue 에서 QPOP 명령어를 통해 방출되는 데이터는 두 BST 의 입력으로 사용된다.
- 하나의 세트는 운동 종목과 장비의 쌍으로 식별하며, 소요 시간과 칼로리는 식별에 사용하지 않는다.
- QPOP 으로 방출된 세트 하나는 두 BST 에 모두 삽입된다. 이때 ExerciseBST 는 운동 종목을 키로 하여 장비를 값으로 저장하고, EquipmentBST 는 장비를 키로 하여 운동 종목을 값으로 저장한다. 즉 같은 쌍을 서로 반대 방향으로 보관한다.
- 삽입하려는 키와 같은 노드가 이미 존재하면 새 노드를 생성하지 않고, 해당 노드의 목록에 값을 추가한다. 키와 값이 모두 일치하는 경우에만 중복으로 판단한다.

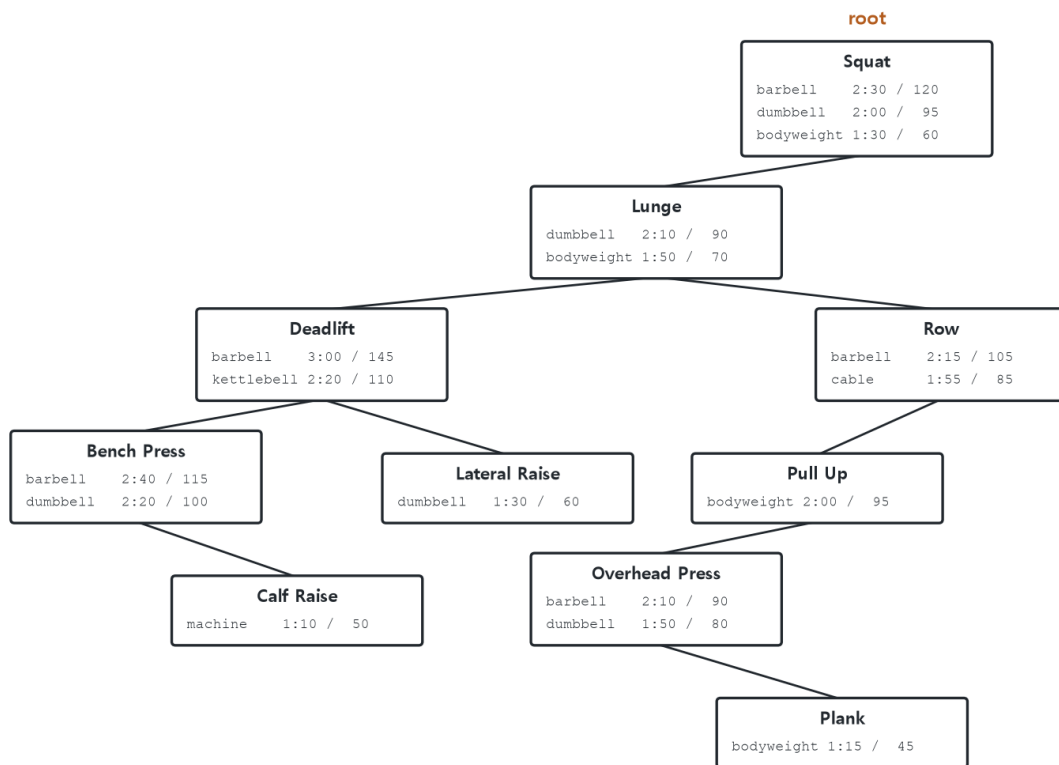


파일에 기록된 순서대로 저장되고, 먼저 저장된 세트가 먼저 방출된다 (FIFO)

[그림 2] WorkoutQueue 예시

2) ExerciseBST

- Queue 에서 방출된 데이터를 이용하여 구축된다.
- BST 의 노드는 ExerciseBSTNode class 로 구현되며, 각 노드는 운동 종목 이름, 해당 운동으로 가능한 장비 목록, 각 세트별 소요 시간과 칼로리 정보를 갖는다.
- BST 에서 지원하는 연산은 삽입, 삭제, 탐색, 출력(중위 순회)이며, 각 연산에 대해서는 Functional Requirements 에서 자세히 설명한다.
- BST 연결 규칙은 다음과 같다.
 - ① 부모 노드보다 운동 종목 이름이 사전순으로 앞서는 경우 왼쪽 서브트리에, 사전순으로 뒤에 오는 경우 오른쪽 서브트리에 위치한다.
 - ② 노드를 삭제할 때 양쪽 자식 노드가 모두 존재하는 경우에는, 오른쪽 서브트리에서 가장 왼쪽에 위치한(사전순으로 가장 앞서는) 노드를 찾아 그 노드가 보유한 모든 정보(키와 세트 목록 전체)를 삭제 대상 노드에 복사한 뒤, 원래의 후계 노드를 제거한다.
 - ③ 한 노드가 보유한 세트가 삭제로 인해 0 개가 되면 해당 노드 자체를 BST 에서 제거한다.

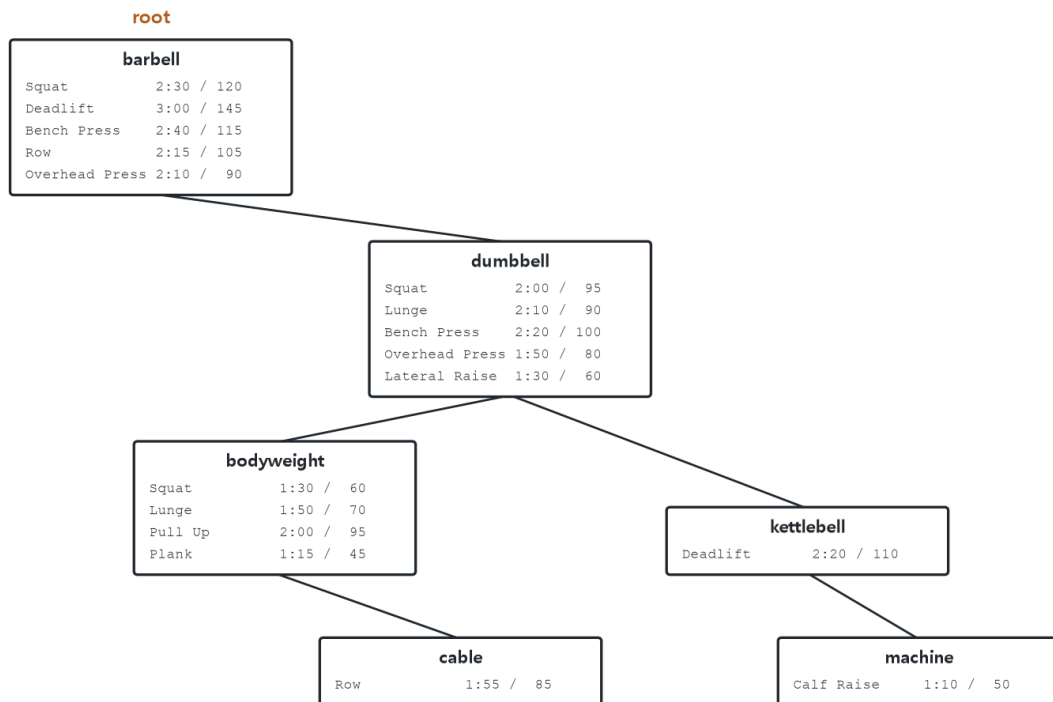


운동 종목 이름을 기준으로 사전순 정렬되며, 각 노드는 그 운동의 모든 세트(장비 · 소요시간 · 칼로리)를 갖는다.

[그림 3] ExerciseBST 예시 (Exercise_List.txt 만 로드한 상태)

3) EquipmentBST

- Queue 에서 방출된 데이터를 이용하여 구축된다.
- BST 의 노드는 EquipmentBSTNode class 로 구현되며, 각 노드는 장비 이름, 해당 장비로 할 수 있는 운동 종목 목록, 각 세트별 소요 시간과 칼로리 정보를 갖는다.
- BST 에서 지원하는 연산은 삽입, 삭제, 탐색, 출력(중위 순회)이며, 각 연산에 대해서는 Functional Requirements 에서 자세히 설명한다.
- BST 연결 규칙은 다음과 같다.
 - ① 부모 노드보다 장비 이름이 사전순으로 앞서는 경우 왼쪽 서브트리에, 사전순으로 뒤에 오는 경우 오른쪽 서브트리에 위치한다.
 - ② 노드를 삭제할 때, 양쪽 자식 노드가 모두 존재하는 경우에는 오른쪽 서브트리에서 가장 작은 이름의 노드를 삭제되는 노드 위치로 이동한다.
 - ③ 한 노드가 보유한 세트가 삭제로 인해 0 개가 되면 해당 노드 자체를 BST 에서 제거한다.

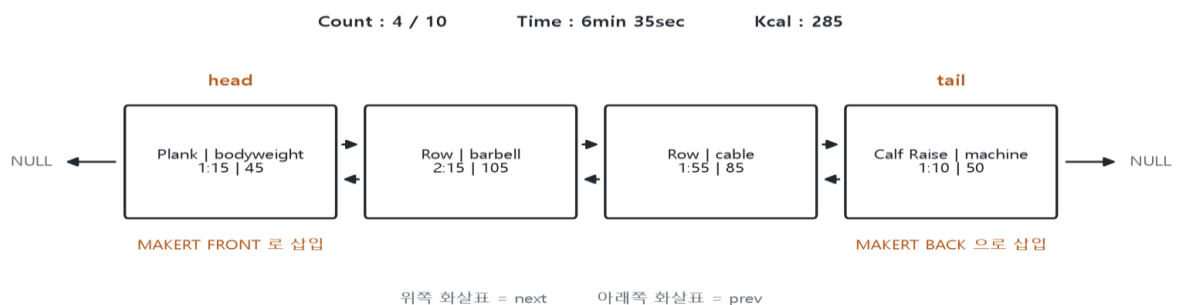


장비 이름을 기준으로 사전순 정렬되며, 각 노드는 그 장비를 쓰는 모든 세트(운동 종목 · 소요시간 · 칼로리)를 갖는다.

[그림 4] EquipmentBST 예시 (Exercise_List.txt 만 로드한 상태)

4) Routine

- Routine 은 BST 에 저장된 운동 데이터를 기반으로 구축된다.
- 리스트의 노드는 RoutineNode class 로 구현되며, 각 노드는 운동 종목, 장비, 소요 시간, 칼로리 정보와 앞뒤 노드를 가리키는 두 개의 포인터를 갖는다. 리스트의 양쪽 끝은 각각 NULL 로 종료된다.
- 리스트에서 지원하는 연산은 앞/뒤 삽입, 앞/뒤 삭제, 탐색 후 삭제, 출력이며, 각 연산에 대해서는 Functional Requirements 에서 자세히 설명한다.
- Routine 을 구성하는 각 노드는 단일 세트의 정보를 담고 있으며, 세트를 추가할 때는 BST 를 탐색하여 데이터를 찾고, 해당 데이터를 바탕으로 새로운 노드를 생성한 뒤 추가한다.
- 세트가 추가될 때 Routine 을 head 부터 tail 까지 확인하여 동일한 세트가 이미 존재하는지 확인하고, 존재하지 않을 경우에만 새 노드를 삽입한다.
- Routine 은 최대 10 세트까지 저장 가능하며, MAKERT 명령어의 위치 인자에 따라 head 쪽 또는 tail 쪽에 노드가 생성되어 연결된다.
- FRONT 옵션으로 여러 세트를 한 번에 추가하는 경우, BST 에서 찾은 순서가 그대로 유지되도록 리스트 앞쪽에 삽입해야 한다. (순서가 뒤집히면 오답으로 처리한다)
- BACK 옵션은 리스트의 tail 뒤에 노드를 붙이며, 여러 세트를 추가하는 경우 BST 에서 찾은 순서대로 차례대로 연결된다.
- tail 에서 노드를 제거하려면 직전 노드를 알아야 하므로, 각 노드는 반드시 이전 노드를 가리키는 포인터를 가져야 한다.
- 리스트中间的 노드를 제거할 때는 제거되는 노드의 앞 노드와 뒤 노드를 서로 직접 연결해야 한다. 이때 앞 노드의 다음 포인터와 뒤 노드의 이전 포인터를 모두 갱신해야 하며, 제거 대상이 head 또는 tail 인 경우에는 head/tail 자체를 갱신한다.
- Routine 에 세트가 추가되거나 제거될 때마다 리스트 전체의 총 소요 시간과 총 칼로리가 즉시 갱신된다.



[그림 5] Routine 예시

□ Functional Requirements

- 출력 포맷은 포맷에 대한 예시일 뿐 실제 출력되는 데이터들과는 차이가 있을 수 있습니다.

명령어	명령어 사용 예 및 기능
LOAD	사용 예) LOAD 텍스트 파일의 데이터 정보를 불러오는 명령어로, 텍스트 파일에 데이터 정보가 존재할 경우 텍스트 파일을 읽어 WorkoutQueue 자료구조에 모두 저장한다. 성공적으로 데이터를 불러온 경우 읽은 데이터를 출력하며, 출력 형태는 "운동종목/장비/소요시간/칼로리"이다. 만약 텍스트 파일이 존재하지 않거나 자료구조에 이미 데이터가 들어가 있으면 에러 코드를 출력한다. 사용되는 텍스트 파일의 이름은 아래와 같으며 파일 이름을 수정하지 않는다. 텍스트 파일: Exercise_List.txt * 데이터 조건 - 운동 종목과 장비 이름은 모두 영문으로 표기해야 한다. - 운동 종목은 대문자로 시작하며, 장비는 소문자로 구성됨. - 운동 종목과 장비가 모두 같은 세트가 중복되어 있는 경우는 없다. - "/"를 구분자로 하여 정보가 저장되어 있다. - 소요 시간은 "x:xx"의 형태로 ':' 문자로 분과 초를 구분한다. - 칼로리는 정수로 저장되어 있다. 출력 포맷 예시) <pre>=====LOAD===== Squat/barbell/2:30/120 Squat/dumbbell/2:00/95 ... ===== =====ERROR===== 100 =====</pre>

ADD	사용 예) ADD Burpee bodyweight 2:45 130 WorkoutQueue 에 데이터를 직접 추가하기 위한 명령어로, 총 4 개의 인자를 추가로 입력한다. 첫 번째 인자부터 운동 종목, 장비, 소요 시간, 칼로리를 나타내며 하나라도 존재하지 않을 시 에러 코드를 출력한다. 이때 LOAD 명령어로 Queue 에 추가한 데이터들과 비교해 같은 세트가 들어올 경우 에러 코드를 출력한다. 이때 같은 세트란 운동 종목과 장비가 모두 일치하는 경우를 말하며, 소요 시간과 칼로리가 달라도 같은 세트로 취급한다. 반대로 장비가 같더라도 운동 종목이 다르면 서로 다른 세트이므로 정상적으로 추가된다. 성공적으로 데이터를 추가하였다면 해당 데이터를 출력하며, 출력 형태는 “운동종목/장비/소요시간/칼로리”이다. 출력 포맷 예시) <pre> =====ADD===== Burpee/bodyweight/2:45/130 ===== =====ERROR===== 200 ===== </pre>
QPOP	사용 예) QPOP WorkoutQueue 에서 데이터를 POP 하여 ExerciseBST & EquipmentBST 를 구성하는 명령어이다. WorkoutQueue 의 front 부터 모든 노드를 POP 하여 해당 노드의 데이터로 자료구조를 구축한다. WorkoutQueue 에 데이터가 존재하지 않을 시 에러 코드를 출력한 뒤 다음 명령어를 계속 수행한다. BST 는 다음과 같은 기준이 적용된다. 1. ExerciseBST: 운동 종목 이름을 기준으로 정렬 - 같은 운동 종목의 세트가 들어오면, 해당 노드의 세트 개수를 count 하는 변수를 +1 하고, 해당 노드에 데이터를 추가 저장한다. 2. EquipmentBST: 장비 이름을 기준으로 정렬 - 같은 장비의 세트가 들어오면, 해당 노드의 운동 개수를 count 하는 변수를 +1 하고, 해당 노드에 데이터를 추가 저장한다. 출력 포맷 예시) <pre> =====QPOP===== Success ===== =====ERROR===== 300 ===== </pre>

SEARCH	사용 예 1) SEARCH EXERCISE Squat 사용 예 2) SEARCH EQUIPMENT dumbbell 사용 예 3) SEARCH SET Deadlift kettlebell BST 에서 세트를 검색하는 명령어이다. 해당 명령어에는 3 개의 옵션이 존재한다. 1. EXERCISE: 운동 종목 이름으로 ExerciseBST 에서 검색을 한다. 이때 해당 운동의 모든 세트가 출력되어야 한다. 2. EQUIPMENT: 장비 이름으로 EquipmentBST 에서 검색을 한다. 이때 해당 장비로 할 수 있는 운동 정보까지 출력되어야 한다. 3. SET: 찾고자 하는 세트의 정보를 “운동종목 장비”의 형식으로 입력하여 검색 후 출력한다. 찾고자 하는 데이터가 검색되지 않는다면 에러 코드를 출력한다. 출력 포맷 예시) 입력: SEARCH EQUIPMENT dumbbell <pre> =====SEARCH===== Squat/dumbbell/2:00/95 Lunge/dumbbell/2:10/90 Bench Press/dumbbell/2:20/100 Overhead Press/dumbbell/1:50/80 Lateral Raise/dumbbell/1:30/60 ===== =====ERROR===== 400 ===== </pre>
MAKERT	사용 예 1) MAKERT BACK EXERCISE Squat 사용 예 2) MAKERT FRONT EQUIPMENT bodyweight 사용 예 3) MAKERT BACK SET Pull Up bodyweight MAKERT 명령어는 BST 에 저장된 데이터를 바탕으로 Routine 에 노드를 추가하는 기능을 수행한다. 첫 번째 인자로 삽입 위치를, 두 번째 인자로 검색 옵션을 입력한다. 1) 삽입 위치 (2 가지) FRONT: Routine 의 맨 앞(head 쪽)에 추가한다. 워밍업 세트를 나중에 앞쪽으로 넣는 경우에 사용한다.

BACK: Routine 의 맨 뒤(tail 쪽)에 추가한다.

FRONT, BACK 이외의 값이 입력되면 에러 코드를 출력한다.

2) 검색 옵션 (3 가지)

EXERCISE: 지정된 운동의 모든 세트를 추가한다.

EQUIPMENT: 특정 장비를 사용하는 모든 세트를 추가한다.

SET: "운동종목|장비" 형식으로 단일 세트를 지정하여 추가한다.

추가 과정에서 지정한 세트의 존재 여부는 먼저 BST 에서 검색되며, 세트가 존재하지 않을 경우 에러 코드를 출력한다. 반대로 성공적으로 추가되면 현재 Routine 의 총 소요 시간과 총 칼로리, 등록된 세트 수가 함께 출력된다.

Routine 은 최대 10 세트까지만 저장 가능하며, EXERCISE 또는 EQUIPMENT 옵션으로 여러 세트를 동시에 추가하는 경우에는 추가 전에 Routine 의 남은 공간을 확인해야 한다. 만약 공간이 부족하다면 해당 요청은 처리되지 않고 에러 코드가 출력된다. 또한 추가하려는 여러 세트 중 하나라도 이미 Routine 에 존재한다면 전체 요청을 에러 코드로 처리하며, 일부만 추가하는 동작은 하지 않는다.

FRONT 옵션으로 여러 세트를 한 번에 추가하는 경우, BST 에서 찾은 순서가 그대로 유지되도록 리스트 앞쪽에 삽입해야 한다. 예를 들어 Squat 의 세 세트를 FRONT 로 추가하면 barbell → dumbbell → bodyweight 순서가 유지되어야 한다.

출력 포맷 예시) 입력: MAKERT FRONT EXERCISE Squat

=====MAKERT=====

Squat/barbell/2:30/120

Squat/dumbbell/2:00/95

Squat/bodyweight/1:30/60

Plank/bodyweight/1:15/45

Row/barbell/2:15/105

Row/cable/1:55/85

Calf Raise/machine/1:10/50

Count : 7 / 10

Time : 12min 35sec

Kcal : 560

=====

=====ERROR=====

500

=====

PRINT	사용 예 1) PRINT EXERCISE 사용 예 2) PRINT EQUIPMENT 사용 예 3) PRINT ROUTINE PRINT 명령어는 BST, Routine 에 저장된 데이터를 출력하는 명령어이다. 이 명령어는 총 3 개의 옵션이 존재한다. 1. EXERCISE: ExerciseBST 를 중위 순회(in-order) 방식으로 탐색하여 데이터를 출력한다. 2. EQUIPMENT: EquipmentBST 를 중위 순회(in-order) 방식으로 탐색하여 데이터를 출력한다. 3. ROUTINE: Routine 을 head 부터 tail 까지 순회하며 모든 노드의 정보를 출력한다. 출력하고자 하는 자료구조가 비어 있는 경우에 에러 코드를 출력한다. 출력 포맷 예시) 입력: PRINT ROUTINE <pre> =====PRINT===== Plank/bodyweight/1:15/45 Row/barbell/2:15/105 Row/cable/1:55/85 Count : 3 / 10 Time : 5min 25sec Kcal : 235 ===== =====ERROR===== 600 ===== </pre>
DELETE	사용 예 1) DELETE FRONT 사용 예 2) DELETE BACK 사용 예 3) DELETE EXERCISE Calf Raise 사용 예 4) DELETE EQUIPMENT cable 사용 예 5) DELETE ROUTINE Squat dumbbell 사용 예 6) DELETE SET Lunge dumbbell DELETE 명령어는 입력된 데이터를 기반으로 해당 세트 또는 운동의 정보를 다양한 자료구조에서 삭제하는 기능을 수행한다. 이 명령어는 총 6 개의 옵션을 제공한다. 1. FRONT: Routine 의 맨 앞 세트를 제거한다. 추가 인자를 받지 않으며, Routine 이 비어 있으면 에러 코드를 출력한다.

	2. BACK: Routine 의 맨 뒤 세트를 제거한다. tail 의 직전 노드를 알아야 하므로 각 노드의 이전 노드 포인터를 이용해야 한다. (이 역시 Routine 이 비어 있으면 에러 코드를 출력한다.) 3. EXERCISE: 지정된 운동의 모든 데이터를 삭제한다. 이 경우 해당 운동의 모든 세트가 ExerciseBST, EquipmentBST, 그리고 Routine 에서 제거된다. 4. EQUIPMENT: 입력된 장비를 사용하는 모든 세트 데이터를 삭제한다. ExerciseBST, EquipmentBST, Routine 에서 모두 제거된다. 5. ROUTINE: 입력된 세트를 Routine 내에서만 삭제한다. 이때 입력 데이터는 "운동종목 장비" 형식을 따른다. BST 에는 영향이 없으며, Routine 에서만 해당 노드가 제거된다. 이때 Routine 의 등록된 세트 수와 총 소요 시간, 총 칼로리가 업데이트된다. 6. SET: 입력된 세트를 단일 데이터로 간주하여 BST(ExerciseBST, EquipmentBST)와 Routine 에서 모두 삭제한다. 위 세 옵션으로 세트가 제거된 결과 어떤 노드의 세트 목록이 0 개가 되면, 해당 노드는 BST 에서 제거된다. 예를 들어 DELETE EXERCISE Calf Raise 를 수행하면 machine 을 사용하는 운동이 없게되므로 EquipmentBST 의 machine 노드도 함께 사라진다. EXERCISE, EQUIPMENT, SET 옵션은 우선적으로 BST 에 저장된 데이터를 삭제하는 것을 목표로 한다. Routine 에 동일한 데이터가 존재할 경우 함께 삭제되지만, 존재하지 않을 경우에는 별도의 오류 처리 없이 BST 에서만 삭제가 이루어진다. delete 작업이 성공적으로 수행되면 다음과 같이 출력 형식을 따른다. 출력 포맷 예시) <pre> =====DELETE===== Success ===== =====ERROR===== 700 ===== </pre>
EXIT	사용 예) EXIT 프로그램상의 메모리를 해제하며, 프로그램을 종료한다. 메모리 누수 시 감점 요인이다. 출력 포맷 예시) <pre> =====EXIT===== Success ===== </pre>

□ 동작 별 에러 코드

동작	에러 코드
LOAD	100
ADD	200
QPOP	300
SEARCH	400
MAKERT	500
PRINT	600
DELETE	700
잘못된 명령어	1000

□ Requirements in implementation

- 모든 명령어는 command.txt 에 저장하여 순차적으로 읽고 처리한다.
- 모든 명령어와 옵션은 반드시 대문자로 입력한다.
- 명령어에 인자(Parameter)가 모자라거나 필요 이상으로 입력 받으면 에러 코드를 출력한다.
- 예외처리에 대해 반드시 에러 코드를 출력한다.
- 메모리 누수 발생 시, 감점 10%
- 잘못된 출력 시, 기능 별 감점
- 출력은 "출력 포맷"을 반드시 따라 한다.
- log.txt 파일에 출력 결과를 반드시 저장한다.
 - log.txt 가 이미 존재할 경우 텍스트 파일 가장 뒤에 이어서 추가로 저장한다.

□ STL 사용 규정

- Queue 자료구조 구현 시 STL queue 사용은 불가하며, 직접 구현해야 한다.
- BST 와 양방향 연결 리스트 또한 직접 구현해야 하며, STL map, set, list, deque 등으로 대체할 수 없다.
- std::string 과 std::vector 는 노드 내부에 데이터를 담는 용도로 자유롭게 사용 가능하다.

□ 구현 시 반드시 정의해야 하는 Class

- WorkoutQueueNode - 운동 세트 정보 Queue 노드 클래스
- WorkoutQueue - 운동 세트 정보 Queue 클래스
- ExerciseBSTNode - 운동 종목을 기준으로 정렬되는 BST 의 노드 클래스
- ExerciseBST - 운동 종목을 기준으로 정렬되는 BST 클래스
- EquipmentBSTNode - 장비를 기준으로 정렬되는 BST 의 노드 클래스
- EquipmentBST - 장비를 기준으로 정렬되는 BST 클래스
- RoutineNode - 양방향 연결 리스트를 구성하는 노드 클래스
- Routine - 양방향 연결 리스트 클래스
- Manager - Manager 클래스
 - 다른 클래스들의 동작을 관리하여 프로그램을 전체적으로 조정하는 역할을 수행

□ Files

- Exercise_List.txt: 프로그램에 추가할 운동 세트 데이터가 저장된 파일
- command.txt: 프로그램을 동작시키는 명령어들을 저장하고 있는 파일
- log.txt: 프로그램 출력 결과를 모두 저장하고 있는 파일

□ 제한사항 및 구현 시 유의사항

- 반드시 제공되는 코드(github 주소 참고)를 이용하여 구현하며 작성된 소스 파일의 이름과 클래스와 함수 이름은 임의로 변경하지 않는다.
- 함수의 반환형과 매개변수는 구현에 맞게 조정할 수 있으며, 클래스의 함수 및 변수는 자유롭게 추가 구현이 가능하다.
- 제시된 Class 를 각 기능에 알맞게 모두 사용한다.
- 프로그램 구조에 대한 디자인이 최대한 간결하도록 고려하여 설계한다.
- 채점 시 코드를 수정해야 하는 일이 없도록 한다.
- 주석은 반드시 영어로 작성한다. (한글로 작성하거나 없으면 감점)
- 프로그램은 반드시 리눅스(Ubuntu 20.04)에서 동작해야 한다. (컴파일 에러 발생 시 감점)
 - 제공되는 Makefile 을 사용하여 테스트하도록 한다.
- 인터넷에서 공유된 코드, AI 로 생성된 코드나 다른 학생이 작성한 코드를 절대 카피하지 않도록 하며, 적발 시 해당 프로젝트 0 점 처리됨을 명시한다.

□ 채점 기준

1. 코드

채점 기준	점수
LOAD 명령어가 정상 동작하는가?	1
ADD 명령어가 정상 동작하는가?	1
QPOP 명령어가 정상 동작하는가?	2
SEARCH 명령어가 정상 동작하는가?	1
MAKERT 명령어가 정상 동작하는가?	1
PRINT 명령어가 정상 동작하는가?	1
DELETE 명령어가 정상 작동하는가?	3
총합	10

- 채점 기준 이외에도 조건 미달 시 감점 (linux 컴파일 에러, 파일 입출력 X, 주석 미흡 등)

2. 보고서

채점 기준	점수
Introduction 을 잘 작성하였는가?	1
Flowchart 를 잘 작성하였는가?	2.5
Algorithm 을 잘 작성하였는가?	3
Result Screen 을 잘 작성하였는가?	2.5
Consideration 을 잘 작성하였는가?	1
총합	10

- 설계의 경우 코드+보고서, 실습의 경우 코드로 점수가 계산됩니다.

□ 제출기한 및 제출방법

3. 제출기한

- 2026 년 10 월 11 일 일요일 23:59:59 까지 클라스(KLAS)에 제출
- (추가 제출: 2026 년 10 월 12 일 월요일 00:59:59 까지, 10% 감점)

4. 제출방법

- 소스코드와 보고서 파일(pdf)을 함께 압축하여 제출
- 확장자가 .cpp, .h, .pdf 가 아닌 파일은 제출하지 않음 (제출시 감점)
- 보고서 파일 확장자가 pdf 가 아닐 시 감점

5. 제출 형식

- 학번_DS_project1.tar.gz (ex. 2026XXXXXX_DS_project1.tar.gz)
- 제출 형식을 지키지 않은 경우 감점

6. 보고서 작성 형식 및 제출 방법

- Introduction: 프로젝트 내용에 대한 설명
- Flowchart: 설계한 프로젝트의 플로우차트를 그리고 설명
- Algorithm: 프로젝트에서 사용한 알고리즘의 동작을 설명
- Result Screen: 모든 명령어에 대해 결과화면을 캡처하고 동작을 설명
- Consideration: 고찰 작성
- 위의 각 항목을 모두 포함하여 작성
- 보고서 내용은 한글로 작성
- 보고서에는 소스코드를 포함하지 않음